

Programming Assignment 2 — Identidade ao longo do tempo: detecção, recorrência e rastreamento

Disciplina	Aprendizado Profundo
Professor	Dario Oliveira
Monitor	Erick Brito
Entrega	02/10, 23h59 — B41254@fgv.edu.br
Apresentação	02/10 - Horários a definir
Formato	Duplas

1. O problema

A aula de detecção começou pelo mapa das tarefas de visão computacional — classificação, localização, detecção, segmentação — e todas elas são definidas sobre **uma imagem**. YOLO e Mask R-CNN entregam, para cada quadro, um conjunto de objetos com caixa, classe e (no caso do Mask R-CNN) máscara. Mas o conjunto do quadro t não tem relação nenhuma com o conjunto do quadro $t + 1$: a ordem das saídas vem do NMS, que ordena por score. O objeto que era o terceiro da lista vira o primeiro quando alguém passa na frente dele.

A aula de RNN deu a máquina para sequências: estado que persiste, BPTT, o problema do gradiente que some, as variantes com portas que existem justamente por causa disso. Só que os exemplos eram sequências de **vetores**, com comprimento e ordem bem definidos. Um vídeo é uma sequência de **conjuntos** de tamanho variável, sem ordem canônica, e o que precisa persistir não é um estado só: é um estado por objeto, com objetos nascendo, sumindo atrás de um poste e voltando trinta quadros depois.

Este PA é a junção. A tarefa de vocês é fazer as arquiteturas das duas aulas produzirem rótulos **identity-aware**: se um objeto aparece no quadro 3 e reaparece no quadro 40, ele tem que sair com o mesmo identificador. Sem usar rastreador pronto (ByteTrack, DeepSORT, OC-SORT, `model.track` do ultralytics e afins estão proibidos).

Isso é possível, e é assim que boa parte da literatura de *multi-object tracking* funciona, mas exige projetar três coisas que a aula não entregou prontas:

1. **o que a recorrência carrega** (o que é o estado de um objeto: caixa, velocidade, aparência);
2. **qual perda otimiza isso** (de onde vem o gradiente?);
3. **como decodificar a previsão em trajetórias** (atribuição, nascimento e morte de tracks).

No PA1 vocês passaram de *class-aware* para *instance-aware* no espaço, aqui, vocês passarão *instance-aware* por quadro para *identity-aware* no tempo. Estão igualmente acopladas.

2. Dados

MOT17 / MOTChallenge. Sequências de pedestres em rua e ambiente interno, com caixas e identidades anotadas quadro a quadro, câmeras paradas e em movimento, densidades muito diferentes entre sequências.

- Download direto, sem conta: <https://motchallenge.net/data/MOT17/>
- O pacote completo tem ~5,5 GB. Existe também o pacote *só de anotações* (~10 MB): comecem por ele. Dá para escrever e depurar a métrica e a associação inteiras antes de baixar um único quadro.
- O benchmark fornece **detecções públicas** (DPM, Faster R-CNN, SDP) por sequência. Isso é o que permite este PA caber em duas semanas: o problema de detecção já vem resolvido, e o trabalho de vocês é o que vem depois dele.
- Formato do `gt.txt`: `frame, id, bb_left, bb_top, bb_width, bb_height, conf, class, visibility`. O campo `visibility` é ouro para a Parte 4.

Split. Por **sequência**, nunca por quadro. Separar quadros aleatoriamente coloca o quadro t no treino e o $t + 1$ na validação, e aí o seu modelo temporal está sendo avaliado em cima de algo que ele praticamente já viu. Reservem pelo menos uma sequência inteira que o modelo nunca viu, e justifiquem o critério na apresentação (câmera parada vs. móvel, densidade, ponto de vista).

3. Regras de engenharia

Permitido: PyTorch; `torchvision.models.detection` (Faster R-CNN, Mask R-CNN) e encoders pré-treinados; `albumations`; `scipy`; `sklearn`; e o código de matching e de métricas que vocês escreveram no PA1.

Permitido com condição: filtro de Kalman de velocidade constante, **apenas** como baseline de comparação. Ele é um baseline honesto e frequentemente difícil de bater; o que não pode é ele ser o modelo temporal da Parte 2. Lá, quem carrega o estado é a rede recorrente.

Proibido:

- rastreadores prontos: SORT, DeepSORT, ByteTrack, OC-SORT, BoT-SORT, Norfair, `motpy`, `trackers` do `supervision`, `model.track()` do `ultralytics`, ou qualquer implementação pronta de associação;
- métricas de rastreamento prontas de biblioteca — `motmetrics`, `TrackEval`, `py-motmetrics`. **Vocês implementam IDF1 e a contagem de ID switches;**
- `torchvision.ops.nms` — **vocês implementam o NMS.**
- clonar uma solução pronta de MOT17. Se usarem ideia de repo ou paper, citem e reescrevam.

O modelo temporal, as perdas, a associação e a gestão de tracks são de autoria de vocês.

Parte 0 — Testes sintéticos

Antes de usar o MOT17, construam um ambiente controlado onde vocês sabem a resposta certa. Três dos quatro artefatos abaixo são reaproveitados nas partes seguintes.

1. **O gerador.** Vídeos 128×128 de 30 a 60 quadros, com 5 a 15 elipses em movimento, tamanhos variados, ruído e contraste variáveis. Exponham como parâmetros pelo menos: número de objetos, velocidade típica e **duração da oclusão**.

Requisito verificável: as elipses precisam ser desenhadas com ordem de profundidade, de modo que uma passe atrás da outra e realmente desapareça, se não é oclusão. Mostrem uma figura com uma trajetória que some por N quadros e volta.

2. **O simulador de detector.** Uma função que recebe as caixas verdadeiras e as estraga de propósito: descarta $p\%$ delas, adiciona ruído nas coordenadas, injeta falsos positivos. Com isso vocês conseguem **validar** a Parte 1 inteira no sintético antes de baixar o MOT17 (Note que isso é exatamente um experimento da Parte 5).
3. **A métrica e seus testes.** Implementem IDF1 e a contagem de ID switches, e mostrem que passam em três casos construídos à mão: (a) predição = ground truth $\Rightarrow$ IDF1 = 1 e zero switches; (b) duas identidades trocadas a partir do quadro $k \Rightarrow$ o número exato de switches que vocês esperam; (c) uma track partida em duas no meio $\Rightarrow$ o efeito esperado em IDF1, que não é o mesmo de (b).
4. **O baseline no piso fácil.** Rodem a associação ingênua da Parte 1 com poucas elipses, lentas e sem oclusão. O IDF1 tem que ficar muito perto de 1.

Depois disso, girem os botões do gerador (mais objetos, mais rápidos, oclusão mais longa) e mostrem onde o baseline começa a quebrar. Esse gráfico é o ensaio da Parte 1.

Parte 1 — Baseline por quadro

1. **Deteções, sem treinar detector.** São duas fontes, e vocês vão usar as duas:

- as **deteções públicas** que acompanham o MOT17 (`det/det.txt`). São três detectores diferentes, DPM, FRCNN e SDP, vocês escolhem **um** como fonte padrão do resto do PA. Digam qual, e por quê;
- um **detector pré-treinado do torchvision** em modo de inferência, classe `person` do COCO.

Fazer fine-tune do detector é opcional e não vale ponto extra por si só — só vale se vocês mostrarem o que isso muda no rastreamento.

2. **Associe o jeito ingênuo:** IoU entre as detecções do quadro t e do quadro $t - 1$, matching guloso ou Hungarian, limiar fixo, ID novo quando nada casa, track morta depois de k quadros sem observação.
3. **Avaliem como trajetórias**, com métricas implementadas por vocês:
 - **IDF1:** exige uma atribuição global um-para-um entre identidades previstas e verdadeiras ao longo da sequência inteira
 - **ID switches** e **fragmentações**, contados explicitamente;
 - **erro de contagem de identidades únicas** no vídeo (o análogo temporal do erro de contagem do PA1).

MOTA é opcional: com o detector congelado, os termos de FP/FN quase não variam entre as configurações de vocês, e ela acaba escondendo a única coisa que muda.
4. Documentem a regra de associação e a gestão de nascimento/morte de tracks. Assim como no PA1, regras diferentes dão números diferentes.
5. **Quantifiquem o fracasso.** O gráfico obrigatório desta parte é o do descolamento, em dois painéis sobre as mesmas sequências: em cima, mAP por quadro e IDF1; embaixo, o número de identidades previstas dividido pelo de verdadeiras e os ID switches por identidade verdadeira. Ordenem as sequências pelo eixo de dificuldade que vocês escolherem: densidade, movimento de câmera, duração de oclusão.

Parte 2 — Memória temporal (escolham UMA trilha)

A fonte de detecções fica **congelada** a partir daqui, o que muda é o que acontece entre os quadros.

Trilha A — RNN como modelo de movimento

Um estado recorrente por track. A cada quadro, o LSTM/GRU recebe a última observação (caixa, opcionalmente confiança e Δt) e prevê a caixa do quadro seguinte; a associação usa IoU entre caixa prevista e caixa observada. Sob oclusão, o estado roda para frente sem observação, e a track sobrevive ou não. Perda $L_1/smooth-L_1$ sobre a caixa, treinada em trajetórias do ground truth; opcionalmente prevejam também uma incerteza e usem log-verossimilhança gaussiana, o que dá um portão de associação adaptativo.

Trilha B — RNN como memória de aparência

Cada detecção vira um embedding D -dimensional (recorte da imagem por um encoder pequeno e pré-treinado). Um agregador recorrente mantém o estado de aparência da track, atualizado a cada observação; a associação é similaridade de cosseno, com um portão geométrico de IoU por cima. Perda contrastiva ou triplet sobre as identidades do ground truth.

A apresentação precisa explicar em que aspecto essas representações (trilha A ou B) melhoram o fracasso da parte 1. Mostrem a comparação com as mesmas métricas lado a lado com a baseline, nas mesmas sequências.

Uma pergunta para responder na apresentação (sem implementar): um vídeo de duas horas não cabe na memória, então a inferência é feita em janelas de T quadros. No PA1, a inferência em mosaico quebrava objetos na fronteira entre tiles. O que quebra aqui, na fronteira entre janelas, e o que a sua representação em particular permitiria fazer para costurar as identidades?

Parte 3 — Ablação

Escolham **um** dos eixos abaixo e rodem a configuração com **3 seeds**, reportando média $\pm$ desvio:

Eixo 1 — a célula recorrente. RNN simples vs. LSTM vs. GRU, no mesmo orçamento aproximado de parâmetros, variando também o comprimento da janela de BPTT truncado ($T \in \{4, 8, 16, 32\}$). A pergunta específica: onde a RNN simples quebra, e isso bate com a história de gradiente que some contada na aula?

Eixo 2 — o regime de treino. *Teacher forcing* $\rightarrow$ *scheduled sampling* $\rightarrow$ *free-running*. Na inferência o modelo se alimenta das próprias previsões (e, sob oclusão, só delas); se ele nunca viu isso no treino, a distribuição muda debaixo dele. Meçam. Incluam *gradient clipping* ligado/desligado e reportem o que acontece sem ele.

Eixo 3 — o que entra na recorrência. Só geometria, só aparência, ou os dois. Qual dos dois sustenta a identidade através de uma oclusão longa, e isso muda com a densidade da cena?

Eixo 4 — direção do contexto. Causal (online, só o passado) vs. bidirecional (offline, com o futuro). Quantos pontos de IDF1 o futuro vale, e a que custo de latência? Reportem os dois números juntos — ganho e atraso.

Parte 4 — Galeria de falhas e horizonte de memória

Três trechos em que o modelo final erra feio, cada um com:

- a figura (tira de quadros com ground truth e predição coloridos por identidade, mais o mapa intermediário relevante - caixa prevista pela recorrência ou matriz de similaridade de embeddings);
- um **diagnóstico**: “esse objeto fica ocluído por 34 quadros; minha janela de BPTT é de 10 e a norma do gradiente cai $20\times$ em 8 passos, então o modelo nunca recebeu sinal de supervisão que atravessasse esse buraco”.

Obrigatório nesta parte: **meçam o horizonte de memória efetivo do seu modelo**, das duas formas:

- Analítica.** A norma de $\partial\mathcal{L}_t/\partial h_{t-k}$ em função de k . É a curva de gradiente que some, dos slides de treinamento de RNN, no seu modelo e nos seus dados. Se fizeram o Eixo 1 da Parte 3, comparem a curva da RNN simples com a do modelo com portas, na mesma janela — os checkpoints já existem.
- Empírica.** Quantos quadros o estado sobrevive a uma oclusão antes de a track morrer ou trocar de ID. Comparem essa distribuição com a distribuição de duração de oclusão do dataset.

E façam **uma correção**: escolham um dos diagnósticos, implementem a mudança que ele sugere, mostrem o antes/depois. Se não funcionar, expliquem o que isso revela sobre o diagnóstico estar errado.

Parte 5 — Teste de estresse (escolham UM)

Os dois são feitos **sem retreinar**, em cima do modelo final.

- Queda de taxa de quadros:** avaliem com o vídeo subamostrado a $1/2$ e $1/5$ da taxa original — curva de degradação do IDF1. Por que um modelo de movimento aprendido em Δt fixo quebra quando Δt muda? Alimentar Δt na recorrência resolveria?
- Qualidade do detector:** degradem as detecções de propósito: descartem $p\%$ delas, adicionem ruído nas caixas, injetem falsos positivos. Façam isso em 3 intensidades diferentes. O modelo temporal absorve ou amplifica a falha do detector? Reportem mAP e IDF1 juntos.

4. Entregáveis

Arquivo	Descrição
Repositório Git	Repositório do trabalho
README.md	Ambiente, download dos dados, um comando que treina, um comando que avalia
metrics.py	Implementação própria de IDF1, ID switches e fragmentações
AI_LOG.md	Ver abaixo
inferencia.ipynb	Recebe o caminho de uma sequência qualquer, devolve o vídeo com as identidades coloridas de forma consistente e a contagem de objetos únicos. Roda sem retreinar
Checkpoint	Pesos do modelo temporal (link se for grande)

Não há relatório escrito. A avaliação é a apresentação. O repositório existe para dar lastro ao que vocês afirmarem lá: toda tabela, curva, imagem e vídeo mostrado na apresentação tem que ser reproduzível a partir dele.

5. Política de uso de IA

Uso de IA é permitido e esperado. O que não é permitido é entregar algo que vocês não entendem.

Subam também um arquivo *AI_LOG.md* que descreve como usaram a IA nesse assignment. Podem citar alguns episódios e como recorreram às ferramentas de IA para resolvê-los.